Nullability в C#

И добавление nullable-типов в существующий код

О чем поговорим

- Nullable reference типы в C#
- Как устроены nullable reference типы
- Поддержка на уровне языка
- Техники и инструменты для внедрения в существующий проект

Nullable reference типы в C#

Reference типы, value типы и null

Language C# Statement(s) .NET Auto Connection <None>

```
string GetValue() => null;

string s = GetValue();

Console.WriteLine($"Length of `{s}`: {s.Length}");
```

Line: 5 Char: 51 Col: 51 LF SPACES

Language C# Statement(s) .NET Auto Connection <None>

NullReferenceException: Object reference not set to an instance of an object. AI Help

```
string GetValue() => null;

string s = GetValue();

Console.WriteLine($"Length of `{s}`: {s.Length}");
```

Line: 5 Char: 1 Col: 1 LF SPACES

Results λ Format Export **Activate Autocompletion**

NullReferenceException

Object reference not set to an instance of an object.

Message	Object reference not set to an instance of an object.
InnerException	null
StackTrace	at UserQuery.Main(), line 5
Data	(0 items)
HelpLink	null
HResult	-2147467261
Source	LINQPadQuery
TargetSite	System.Reflection.MethodBase

```
string result = s is null ?
    "String is null" :
    $"Length of `{s}`: {s.Length}";

Console.WriteLine(result);
```

Как понять в существующем коде где нужны проверки?

- Для value типов довольно просто: `int? bool? long? DateTime?`
- `?` указывает на возможность `null` в качестве значения
- Компилятор превращает nullable value тип в **`Nullable<T>`**
- Для reference типов не всегда просто понять где нужна проверка в коде

```
DateTime? GetValue() => null;

DateTime? dt = GetValue();

string result = dt.HasValue ?
    "String is null" : $"Length of `{s}`: {s.Length}";

Console.WriteLine(result);
```

Nullable reference типы (NRT)

- Все reference типы в C# – nullable
  - `string s = GetValue(); // s – nullable тип`
- Начиная с C# 8 поменяли подход:
  - Типы стали non-nullable по умолчанию
  - В язык добавили синтаксис для указания на nullable типы (аннотации)
  - Ответственность за null-safety ложится на плечи разработчика при дизайне кода
  - Проверки на этапе компиляции

```
string s = GetValue();
Console.WriteLine($"Length of `{s}`: {s.Length}");

string? GetValue() => null;
```

CS8632 The annotation for nullable reference types should only be used in code within a '#nullable' annotations context.

Анализ выполнения кода (flow analysis)

- IDE и компилятор анализирует путь выполнение кода
  - Определяют warning/no warning
- Использование `var` всегда расценивается как `nullable`
  - Пока анализ выполнения кода не определил обратное
- Операторы `??` (null-coalescing) и `!` (null-forgiving)
  - Подавляют warnings
  - Могут помочь с направлением анализа при миграции на null reference типы
  - …но должны рассматриваться как **анти-паттерн**

Nullable reference типы в C#

- Отсутствует безопасность в runtime
- Проверки на уровне дизайна кода и этапе компиляции помогают определить может ли переменная быть `null` перед тем как выполнить обращение к значению (`dereferencing`)
- Помогают с анализом выполнения кода

Что внутри

Value types vs. reference types

- Компилятор превращает value типы в `Nullable<T>`
- Reference типы всегда были и остаются nullable 😞
- Как компилятор и IDE понимают о nullability?

```
#nullable disable
string GetString1() => "";


.method private hidebysig instance string
  GetString1() cil managed
{
  .maxstack 8

  IL_0000: ldstr        ""
  IL_0005: ret

}
```

Начиная с C# 8

```
#nullable enable
string? GetString2() => "";

.method private hidebysig instance string
  GetString2() cil managed
{
  .custom instance void
System.Runtime.CompilerServices.NullableContextAttribute::.ctor([in] unsigned int8)
    = (01 00 02 00 00 ) // .....
    // unsigned int8(2) // 0x02
  .maxstack 8

  IL_0000: ldstr        ""
  IL_0005: ret

}
```

Значения атрибута `NullableContextAttribute`

- Неопределенность: 0
  - По умолчанию, до C# 8 (все типы могут быть null)
- Без аннотации: 1
  - Любой reference тип – не nullable по умолчанию
- С аннотацией: 2
  - Любой reference тип неявно помечен как ?
- …

См. также `NullableAttribute`

Включение null reference типов в проекте

- Блок кода,файл или проект – контекст для nullable аннотаций
- В коде пишем `#nullable value`
- В проекте `<Nullable>value</Nullable>`
- Возможные значения:
  - `disable` – до C# 8.0
  - `enable` – все проверки и возможности языка
  - `warnings` – проверки и warnings при обращении к значению и вероятности `null`
  - `annotations` – нет анализа, но позволяет использовать ?

Какой контекст аннотации использовать

- В новом проекте: `enable`
- Начиная с .NET 6 – по умолчанию `enable`
- При переходе на NRT – зависит от проекта и команды
  - disable на уровне проекта, enable на уровне файлов пока не завершена миграция
  - enable на уровне проекта, жизнь с warnings пока они не исправлены
  - warnings на уровне проекта, enable на уровне файлов
  - annotations на уровне проекта для начала использования ? без анализа кода

Treat nullable reference warnings as errors

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
4      <TargetFramework>net6.0</TargetFramework>
5      ..
6      <Nullable>enable</Nullable>
7      <WarningsAsErrors>Nullable</WarningsAsErrors>
8    </PropertyGroup>
9
10 </Project>
```

Помечаем nullable типы в C#

Достаточно ли оператора ?

```
#nullable enable

public static string? Slugify(string? value)
{
    if (value is null)
    {
        return null;
    }

    return value.Replace(" ", "-").ToLowerInvariant();
}

void Main()
{
    string? slug = Slugify("this is fine");

    Console.WriteLine(slug.Length);
}
```

CS8602 Dereference of a possibly null reference.

Уточняющие аннотации

```
using System.Diagnostics.CodeAnalysis;

#nullable enable

[return: NotNullIfNotNull("value")]
public static string? Slugify(string? value)
{
    if (value is null)
    {
        return null;
    }

    return value.Replace(" ", "-").ToLowerInvariant();
}

void Main()
{
    var slug1 = Slugify(null);
    var slug2 = Slugify("this is fine");

    Console.WriteLine(slug1.Length);
    Console.WriteLine(slug2.Length);
}
```

JetBrains.Annotations

- JetBrains поддерживает Nuget-пакет
- Многие библиотеки включают его в свой код
  - Entity Framework Core, Unity, Hangfire, …
- К сожалению, только для ReSharper и JetBrains Rider :(
  - `[CanBeNull], [NotNull]`
  - `[ItemNotNull], [ItemCanBeNull]`
  - `[ContractAnnotation]`
- Примеры использования Enhance ASP.NET Projects With JetBrains Annotations

Nullability и дженерики

Используйте ? где нужно:

```
List<string>
List<string?>
List<string>?
```

Используйте аннотации

```
[return: MaybeNull]
public T Find<T>(IEnumerable<T> sequence, Func<T, bool> predicate)
```

Используйте `notnull` ограничение

```
public static void WriteToConsole<T>(T item) where T : notnull
WriteToConsole(null); // CS8714 doesn’t match ‘notnull’ constraint
```

Referenced код, библиотеки и фреймворки

- Использование Nullable Reference типов не вызывает сложностей если ваши зависимости тоже их используют
  - .NET BCL проаннотирована (начиная с C# 8)
  - Некоторые OSS библиотеки тоже, многие нет !!
    - Предполагаем, что могут возвращаться null
    - У R# и Rider есть пессимистический анализ (pessimistic analysis)
- Аннотирование разрабатываемой библиотеки или кода, который будут использовать другие разработчики – хорошая практика
  - Аннотации предоставляют design-time и compile-time подсказки (хинты)
- 😞 Nullable Reference Types не гарантируют безопасность в рантайме (runtime safety)
  - Все еще нужно выполнять проверки на null **в runtime**
  - Пользователи библиотеки/вашего кода могут не использовать C# или у них могут быть отключены проверки

TLDR; Аннотации

- ? – не единственная аннотация
- Помогают анализатору кода с пониманием выполнения программы
- Аннотации компилятора и JetBrains.Annotations – хорошая связка
- При разработке библиотек / sharable кода все еще нужны проверки на `null`

С чего начать

- Классы с небольшим или нулевым количеством зависимостей на другие
  - Часто: DTOs / POCOs
- Обычно используются большим количеством зависимостей
  - Меняем в одном месте → влияние на несколько
- Как найти такие классы
  - ReSharper Type Hierarchy Diagram
  - Инструменты архитектуры в Visual Studio
  - NDepend
  - …
- Иногда редизайн класса/компонента/проекта даст лучшие результаты

Аннотация класса

- Включаем `#nullable enable`
- Для каждого свойства и параметра конструктора
  - Находим все места, где происходит установка значения
  - Если есть вероятность `null`, используем оператор `?`

Как жить с `null` в C# коде

- Помнить, что полностью избавиться от `null` в проекте не получится
- Использовать инструменты, чтобы быть более уверенными в своем коде – отсутствие логических ошибок при обработке отсутствия значений, `NullRereferenceException`

Трекинг значения

TrackerBase.cs

GoogleAnalyticsTracker.Core (net6.0) | GoogleAnalyticsTracker.Core.TrackerBase | TrackerBase(string trackingAccount, IAnalyticsSessio

```
49      {
50      }
51
52      public TrackerBase(string trackingAccount, IAnalyticsSession analyticsSession, I
53      {
54          TrackingAccount = trackingAccount;
55          AnalyticsSession = analyticsSession;
56
57          EndpointUrl = GoogleAnalyticsEndpoints.Default;
```

100 % | No issues found | Ln: 54 | Ch: 17 | SPC | CRLF

Inspection Results - Origin of 'property TrackingAccount'

Origin of 'property TrackingAccount'

Context: Type and Member

Type to search

- property TrackingAccount
  - TrackingAccount = **trackingAccount** in [TrackerBase.TrackerBase]
    - base(**optionsAccessor.Value.TrackerId**, ...) in [AspNetCoreTracker.AspNetCore
      - options.TrackerId = **"UA-XXXXXX-XX"** in [Startup.ConfigureServices]
      - TrackerId = **default!** in [GoogleAnalyticsTrackerOptions.TrackerId]
    - base(**trackingAccount**, ...) in [SimpleTracker.SimpleTracker]
    - this(**trackingAccount**, ...) in [TrackerBase.TrackerBase]
      - base(**trackingAccount**, ...) in [SimpleTracker.SimpleTracker]

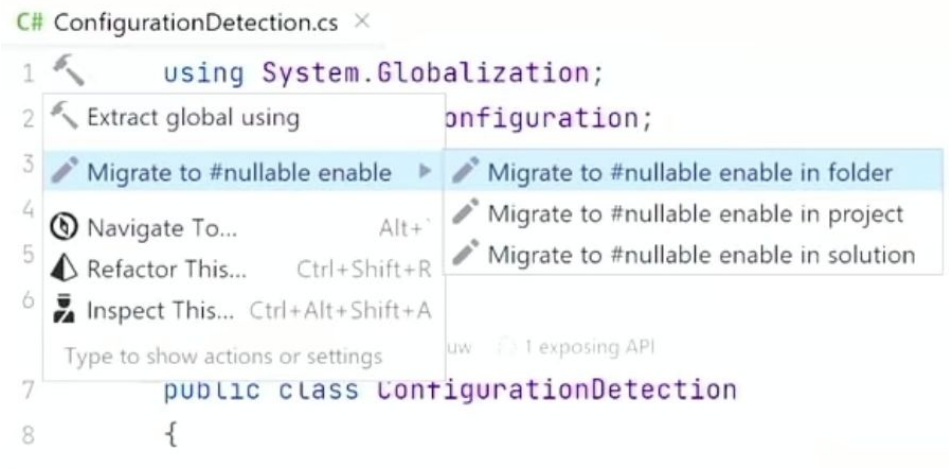
```
public void ConfigureServices(IServiceCollect:
{
    services.AddRouting(options => options.Lo

    services.Configure<CookiePolicyOptions>(o
    {
        // This lambda determines whether use
        options.CheckConsentNeeded = _ => tru
        options.MinimumSameSitePolicy = SameS
    });

    services.AddGoogleAnalyticsTracker(option
    {
        // ReSharper disable once StringLiter
        options.TrackerId = "UA-XXXXXX-XX";
        options.ShouldTrackRequestInMiddlewar
```

Автоматическая миграция*

- Добавляет `[NotNull]` и `[CanBeNull]` начиная с базовых классов
- Выводит аннотации на основе использования



(Де)сериализация JSON

```
using System.Text.Json.Serialization;

#nullable enable

public class WeatherForecastWithPropertyName
{
    public DateTimeOffset Date { get; set; }
    public int TemperatureCelsius { get; set; }

    [JsonPropertyName("summary")]
    public string Summary { get; set; }

    [JsonPropertyN
    public int Wir
}
```

CS8618 Non-nullable property 'Summary' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

Можем попытаться исправить так если уверены что поле всегда не null 🤔

```
[JsonPropertyName("summary")]
public string? Summary { get; set; }
```

```
[JsonPropertyName("summary")]
public string Summary { get; set; } = default!; // анти-паттерн, обманываем компилятор
```

```
[JsonPropertyName("summary")]
public string Summary { get; set; } = "Missing"; // OK
```

```
using System.Diagnostics.CodeAnalysis;
using System.Text.Json.Serialization;
using Newtonsoft.Json;

#nullable enable

public class WeatherForecastWithPropertyName
{
    public DateTimeOffset Date { get; set; }
    public int TemperatureCelsius { get; set; }

    [JsonPropertyName("wind_speed")]
    public int WindSpeed { get; set; }

    [AllowNull]
    private readonly string _summary; // OK, но больше кода

    [JsonProperty("summary")]
    public string Summary
    {
        get => _summary;
        init => _summary = value ?? "Missing";
    }
}
```

C# 11

Language C# Statement(s) .NET 9.0 Connection <None>

```
using System.Text.Json.Serialization;

#nullable enable

public class WeatherForecastWithPropertyName
{
    public DateTimeOffset Date { get; set; }
    public int TemperatureCelsius { get; set; }

    [JsonPropertyName("summary")]
    public required string Summary { get; set; }

    [JsonPropertyName("wind_speed")]
    public int WindSpeed { get; set; }
}
```

Entity Framework Core

Свойство модели является опциональным, если `null` – валидное значение для него. Если `null` – не валидное значение при присваивании, то свойство рассматривается как обязательное (`required`).

При маппинге на схему БД обязательные свойства становятся non-nullable колонками, а опциональные – как nullable колонки.

⚠️ При активации Null Reference типов в проекте, схема БД может измениться неожиданным образом. Обращайте внимание на миграции!

EF Core: Working with Nullable Reference Types

А как в Kotlin?

- В **Kotlin** «null-безопасность» — фундаментальная часть языка: функции, классы и библиотеки стараются максимально использовать non-nullable типы и вынуждать разработчика явно обрабатывать ситуации, в которых значение может быть null.
- **Kotlin** изначально создавался с концепцией «большинство типов — non-nullable». Чтобы объявить переменную, которая *может* быть `null`, нужно явно указать тип с `?`, например: `String?`, `Int?` и т.д. Если тип не содержит `?`, значит null ему не присваивается.
- В **Kotlin** попытка присвоить null в переменную non-nullable типа вызывает *ошибку* компиляции.
- **Kotlin** ориентирован на взаимодействие с Java-кодом (через JVM). При вызовах Java-методов, у которых нет информации о `null`, Kotlin «выводит» так называемые *platform types* (например, `String!`), где компилятор не может гарантировать отсутствие `null`. В таких случаях ответственность за проверку переходит к программисту.